

Mathematical Background and Implementation Guide

n2_theory_index.py

FORM and LiE computation of four-dimensional $\mathcal{N} = 2$ indices
Simple and semisimple product gauge groups

Index/cache revision: 10 September 2026

Purpose. The canonical implementation computes the flavor-unrefined right superconformal index of a Lagrangian four-dimensional $\mathcal{N} = 2$ gauge theory. FORM performs exact truncated series algebra. LiE decomposes individual Adams operations and intermediate products. Python pairs dual irreps to extract the final singlet, independently for every simple gauge factor. SQLite stores exact decompositions and final singlet coefficients. Sage supplies input/representation data, the dual-label convention and the exact returned polynomial ring.

This revision updates the cache, projection algorithm, helper descriptions and validation. The unrevised index convention, matter characters, FORM expansion, JSON examples and interpretation are retained. Earlier validation is dated explicitly. The companion implementation reference provides package-wide context.

Contents

1 Scope and program architecture	2
2 Index convention and single-letter functions	2
3 Matter characters for simple and product groups	3
4 Plethystic exponential and exact cutoffs	3
5 How FORM represents and expands the index	3
6 Adams operations and LiE decomposition	5
7 Gauge-singlet projection for product groups	6
8 The SQLite character and singlet cache	8
9 Python functions and their mathematical roles	9
10 JSON examples and command-line use	10
11 Validation and representative performance	11
12 Interpretation and limitations	12
References	13

1 Scope and program architecture

The supported gauge algebra is a finite semisimple algebra

$$\mathfrak{g} = \bigoplus_{a=1}^s \mathfrak{g}_a, \quad G = \prod_{a=1}^s G_a, \quad (1)$$

where every $\mathfrak{g}_a$ has Cartan type $A_r, B_r, C_r, D_r, E_6, E_7, E_8, F_4$, or G_2 . The compact group convention is the simply connected one used by the shared Lie-algebra module; in particular C_r is written $\mathrm{Sp}(r)$. Abelian gauge factors are outside the current input model.

The calculation has four layers:

1. `check_input_data` validates the existing simple- or product-group JSON convention and resolves names to Dynkin labels.
2. `_build_form_program` constructs the truncated plethystic exponential with formal character symbols.
3. `CharacterDecompositionCache` asks LiE for the necessary virtual character decompositions and caches decompositions and singlet coefficients in SQLite.
4. `_project_terms` extracts the gauge singlet factor by factor and `_to_sage_polynomial` packages the exact answer.

This division is important: FORM never needs to know a root system, while LiE never needs to manipulate the fugacities t, y, u .

2 Index convention and single-letter functions

Choose the supercharge defining the right index. With the conventions of Gadde, Pomoni, and Rastelli, the BPS condition and trace are [1]

$$\delta_R = \Delta - 2\bar{j} - 2R + r = 0, \quad (2)$$

$$\mathcal{I}(t, y, v) = \mathrm{Tr}_{\delta_R=0} (-1)^F t^{2(\Delta+\bar{j})} y^{2j} v^{-r-R}. \quad (3)$$

The code substitutes

$$v = u^2, \quad (4)$$

so every computational power of u is integral.

The vector and half-hypermultiplet single-letter indices are [1]

$$f_V(t, y, u) = \frac{t^2 u^2 - t^4 u^{-2} - t^3(y + y^{-1}) + 2t^6}{(1 - t^3 y)(1 - t^3 y^{-1})}, \quad (5)$$

$$f_H(t, y, u) = \frac{t^2 u^{-1} - t^4 u}{(1 - t^3 y)(1 - t^3 y^{-1})}. \quad (6)$$

The denominator counts the two compatible derivatives:

$$\frac{1}{(1 - t^3 y)(1 - t^3 y^{-1})} = \sum_{p, q \geq 0} t^{3(p+q)} y^{p-q}. \quad (7)$$

FORM uses a finite rectangular sum with $0 \leq p, q \leq \lfloor D/3 \rfloor$. Terms whose combined t degree exceeds D are removed automatically.

3 Matter characters for simple and product groups

For a product group, an irreducible external tensor product is

$$\mathbf{R}_h = \boxtimes_{a=1}^s R_{h,a}, \quad \chi_{\mathbf{R}_h}(g_1, \dots, g_s) = \prod_{a=1}^s \chi_{R_{h,a}}(g_a). \quad (8)$$

An omitted representation in the JSON input is the singlet, whose character is one.

The adjoint representation of the product algebra is a direct sum,

$$\text{ad}_{\mathfrak{g}} = \bigoplus_{a=1}^s (\mathbf{1}, \dots, \text{ad}_{\mathfrak{g}_a}, \dots, \mathbf{1}), \quad \chi_{\text{ad}_{\mathfrak{g}}} = \sum_{a=1}^s \chi_{\text{ad}_a}. \quad (9)$$

This explains why vector-multiplet characters are added, whereas a bifundamental hypermultiplet character is multiplied across its gauge factors.

For multiplicity N_h , the character measured in half-hyper units is

$$\hat{\chi}_h = \begin{cases} N_h \chi_{\mathbf{R}_h}, & \text{half hyper,} \\ N_h (\chi_{\mathbf{R}_h} + \chi_{\overline{\mathbf{R}}_h}), & \text{full hyper.} \end{cases} \quad (10)$$

Conjugation is simultaneous and factorwise:

$$\overline{\mathbf{R}}_h = \boxtimes_{a=1}^s \overline{R}_{h,a}. \quad (11)$$

Thus a full hyper in $R_1 \boxtimes R_2$ contains $R_1 \boxtimes R_2$ and $\overline{R}_1 \boxtimes \overline{R}_2$; it does not contain the two mixed tensor products unless they are separate matter entries.

4 Plethystic exponential and exact cutoffs

With flavor fugacities unrefined, define the representation-valued letter index

$$\mathcal{L}(t, y, u; g) = f_V(t, y, u) \sum_{a=1}^s \chi_{\text{ad}_a}(g_a) + f_H(t, y, u) \sum_h \hat{\chi}_h(g). \quad (12)$$

The free-field index before gauge projection is the plethystic exponential [1, 2]

$$\mathcal{Z}(t, y, u; g) = \text{PE}[\mathcal{L}] = \exp E(t, y, u; g), \quad (13)$$

$$E(t, y, u; g) = \sum_{j=1}^{\infty} \frac{1}{j} \mathcal{L}(t^j, y^j, u^j; g^j). \quad (14)$$

Since every letter begins at t^2 , a result through t^D needs only

$$1 \leq j \leq M = \left\lfloor \frac{D}{2} \right\rfloor. \quad (15)$$

The same lower bound implies that only $E^k/k!$ with $k \leq M$ can contribute. Consequently both the Adams sum and the ordinary exponential have finite, exact cutoffs at the requested order.

5 How FORM represents and expands the index

5.1 Formal character symbols

Each distinct pair “gauge-factor position, Dynkin labels” receives a commuting FORM function $C_p(j)$. Mathematically it denotes the j -th Adams operation

$$C_p(j) = \psi^j(\chi_{R_p}), \quad (\psi^j \chi)(g) = \chi(g^j). \quad (16)$$

For a bifundamental, FORM therefore sees $C_p(j)C_q(j)$. It expands only the coefficients and products of these formal objects; their Lie-theoretic decomposition is postponed until after the series is complete.

At Adams degree j , `_build_form_program` constructs

$$\mathcal{L}_j = f_V(t^j, y^j, u^j) \sum_{p \in \mathcal{V}} C_p(j) + f_H(t^j, y^j, u^j) \sum_m N_m \prod_{p \in m} C_p(j), \quad (17)$$

where $\mathcal{V}$ contains the adjoint character of every gauge factor and the matter monomials already include the full-hyper conjugate term from Eq. (10).

5.2 The generated FORM program in plain language

The central generated block has the following form:

Listing 1: Schematic FORM program generated by Python.

```
S m,n,y,idx1,idx2,j,z,u,t(:D);
CF d,C0,C1,...;
PolyRatFun d;
Function Kvec,Khyp;

L J=sum_(idx1,0,D/3,m^idx1)
    *sum_(idx2,0,D/3,n^idx2);
id m=t^3*y;
id n=t^3/y;
.sort

L itotal=sum_(j,1,D/2,(vector terms + hyper terms)/j);
id Kvec(t?,y?,u?)=J*(t^2*u^2-t^4/u^2-t^3*y-t^3/y+2*t^6);
id Khyp(t?,y?,u?)=J*(t^2/u-t^4*u);
.sort
```

FORM construct	Meaning in this calculation
<code>S ... t(:D);</code>	Declares commuting symbols. The maximum-power declaration automatically discards $t^{D+1}, t^{D+2}, \dots$ during term normalization [4, Sec. 7.143].
<code>CF d,C0,...;</code>	Declares commuting functions. The $C_p(j)$ objects are formal Adams-operated characters.
<code>PolyRatFun d;</code>	Makes <code>d(numerator,denominator)</code> the exact rational coefficient container, so equal monomials combine without floating point arithmetic [4, Sec. 7.114].
<code>L name=...;</code>	Creates a local FORM expression. <code>J</code> , <code>itotal</code> , <code>I</code> , and <code>result</code> are successive exact expressions.
<code>sum_(j,a,b,X)</code>	Generates the finite sum $\sum_{j=a}^b X$; the last argument is the summand [4, Sec. 8.67].
<code>id lhs=rhs;</code>	Applies an algebraic substitution. A question mark denotes a wildcard, so <code>Kvec(t?,y?,u?)</code> matches arguments such as <code>Kvec(t^j,y^j,u^j)</code> [4, Secs. 7.70–7.71].
<code>.sort</code>	Ends a FORM module, expands products, combines identical terms, and enforces the t cutoff before the next module [4, Sec. 2.3].

5.3 Why the auxiliary symbol z gives the exponential

Let $E = \text{itotal}$ and $M = \lfloor D/2 \rfloor$. The code begins with $I = zE$. For $k = 2, \dots, M$ it applies

$$z \longmapsto 1 + \frac{zE}{k}. \quad (18)$$

After the k -th step, induction gives

$$I_k = \sum_{r=1}^{k-1} \frac{E^r}{r!} + z \frac{E^k}{k!}. \quad (19)$$

The final substitution $z \mapsto 1$, followed by adding the vacuum term, yields

$$1 + I_M = \sum_{r=0}^M \frac{E^r}{r!} = \exp(E) + O(t^{D+1}). \quad (20)$$

This is an ordinary exponential of a representation-valued series; Bose and Fermi statistics have already entered through the signed single-letter functions.

6 Adams operations and LiE decomposition

6.1 The representation-ring calculation

For each irrep R and exponent vector $\mathbf{n} = (n_1, n_2, \dots)$, the cache stores the virtual decomposition

$$D_R(\mathbf{n}) = \text{Decomp} \left[\bigotimes_{j \geq 1} (\psi^j R)^{\otimes n_j} \right]. \quad (21)$$

An Adams operation scales every weight by j and can produce a virtual representation with negative coefficients [3, Secs. 3.4, 4.5]. For the A_1 fundamental,

$$\psi^2(\mathbf{2}) = \mathbf{3} - \mathbf{1}, \quad (22)$$

which LiE prints as a signed decomposition polynomial.

6.2 The LiE commands in plain language

LiE expression	Meaning
<code>Adams(2, [1], A1)</code>	Decomposes ψ^2 of the A_1 irrep with label [1]; the result is $-1X[0]+1X[2]$ [3, Sec. 4.5].
<code>tensor([1], [1], A1)</code>	Returns $1X[0]+1X[2]$ for $V_{[1]} \otimes V_{[1]}$ [3, Sec. 4.5].
<code>tensor(p, q, A1)</code>	Tensors two decomposition polynomials. The revised projector uses this for intermediate products only.
<code>tensor(p, q, [0], A1)</code>	LiE's targeted trivial coefficient operation. The revised projector instead performs the final dual-irrep pairing in Python.
<code>maxnodesN/maxobjectsN</code>	Raises LiE node/object limits. The code retries with these settings only after a failed ordinary run, avoiding eager large-allocation startup cost [3, Sec. 2.3].

A decomposition polynomial has the form

$$p = \sum_{\lambda \in P_+} m_\lambda X[\lambda_1, \dots, \lambda_r], \quad m_\lambda \in \mathbb{Z}. \quad (23)$$

The parser preserves the signed integer m_λ . It also removes LiE's optional “New tree space with maximum number of nodes” notification by recognizing the message prefix, rather than by deleting a fixed number of output characters.

7 Gauge-singlet projection for product groups

The gauge index is the Haar projection

$$\mathcal{I}(t, y, u) = \int_G d\mu_G(g) Z(t, y, u; g). \quad (24)$$

Equivalently, it is the multiplicity of the trivial representation in every coefficient. After FORM expansion, a typical formal-character monomial is

$$\mathcal{C} = \prod_p \prod_{j \geq 1} C_p(j)^{n_{p,j}}. \quad (25)$$

If $a(p)$ is the gauge factor associated with character p , define the virtual G_a representation

$$X_a(\mathcal{C}) = \bigotimes_{p: a(p)=a} \bigotimes_{j \geq 1} (\psi^j R_p)^{\otimes n_{p,j}}. \quad (26)$$

Because both the group and normalized Haar measure factorize,

$$d\mu_G = \prod_{a=1}^s d\mu_{G_a}, \quad (27)$$

$$\text{mult}_{\mathbf{1}_G}(\mathcal{C}) = \prod_{a=1}^s \text{mult}_{\mathbf{1}_a}(X_a(\mathcal{C})). \quad (28)$$

This is the precise rule implemented by `_project_terms`. It partitions a FORM term by gauge-factor position, batches distinct character-product singlet requests for each factor, and multiplies the returned integers. Empty character products have singlet multiplicity one. SQLite is checked before decomposition.

7.1 Exact pairing from character orthogonality

For compact G and normalized Haar measure,

$$\int_G \chi_\lambda(g) \overline{\chi_\mu(g)} d\mu(g) = \delta_{\lambda\mu}, \quad \chi_{\mu^*} = \overline{\chi_\mu}. \quad (28a)$$

Thus for two partial virtual characters,

$$A = \sum_{\lambda} a_{\lambda} \chi_{\lambda}, \quad B = \sum_{\mu} b_{\mu} \chi_{\mu}, \quad \boxed{[1](AB) = \sum_{\lambda} a_{\lambda} b_{\lambda^*}}. \quad (28b)$$

Character orthogonality is Corollary 35.9 of [6]. Equation (28b) follows by substitution and linearity, including negative coefficients. Equivalently, $(V \otimes W)^G \simeq \text{Hom}_G(V^*, W)$; the latter is the space of linear maps commuting with the group action. Schur's lemma selects dual irreps. The singlet functional here is bilinear; an ordinary coefficient dot product would incorrectly pair complex irreps with themselves.

7.2 Split before decomposition

A symbolic input $(\text{labels}, (n_1, n_2, \dots))$ stands for $\prod_j \psi^j(R)^{n_j}$. The revised procedure is:

1. Remove trivial-character factors, since $\psi^j(\mathbf{1}) = \mathbf{1}$.
2. Expand exponents into individual Adams factors and split them into two approximately balanced parts A and B .
3. Fully decompose the needed *partial* products, reusing persistent and in-memory entries. Individual Adams operations remain indivisible.
4. Apply (28b) using sparse dictionary lookups and save the resulting integer.

The splitter's cost estimate for an atom is

$$c(\lambda, j) = j \max \left(1, \sum_i \lambda_i \right). \quad (28c)$$

It places larger estimates first on the currently lighter side, preferring an already-present base irrep when the two costs tie. This source-derived heuristic balances approximate highest-weight sizes, not exact tensor complexity, and carries no optimality guarantee. Repeated factors of one base irrep can be split across the two sides before their complete product is decomposed.

For example, $\psi^4(\text{adj})\psi^5(\text{adj})$ needs only the two individual Adams expansions and their pairing. In contrast, $\psi^3(\text{adj})^3$ leaves the intermediate $\psi^3(\text{adj})^2$, which can remain expensive. The full decomposition API is still available when explicitly requested; the singlet path avoids tensor-decomposing the complete product.

7.3 Dual labels and signed examples

The highest weight of the dual irrep is

$$\lambda^* = -w_0\lambda. \quad (28d)$$

The map permutes fundamental weights [6, Prop. 32.1]. `_dual_permutation` obtains that permutation with the existing Sage duality helper and caches it; `_singlet_pair` applies it to label tuples. See also Sage's `dual()` and `inner_product()` [7]. The pairing iterates over the smaller support, using $O(r \min(N_A, N_B))$ label operations. The costly step can still be constructing A or B .

For $SU(3)$, $(a, b)^* = (b, a)$, so $[\mathbf{1}](\mathbf{3} \otimes \overline{\mathbf{3}}) = 1$ but $[\mathbf{1}](\mathbf{3} \otimes \mathbf{3}) = 0$. Actual conjugate orientations must remain in cache keys, even where theory enumeration identifies conjugates. For $SU(2)$, $\mathbf{2} \otimes \mathbf{2} = \mathbf{1} \oplus \mathbf{3}$ implies $[\mathbf{1}](\mathbf{2}^{\otimes 4}) = 1^2 + 1^2 = 2$. Pseudoreality adds no sign to this pairing. Signed Adams coefficients are different: from (22), $[\mathbf{1}](\psi^2 \mathbf{2}) = -1$, while $[\mathbf{1}](\psi^2 \mathbf{2})^2 = (-1)^2 + 1^2 = 2$.

8 The SQLite character and singlet cache

For the exponent vector in Eq. (21), define its total Adams order

$$q(\mathbf{n}) = \sum_{j \geq 1} j n_j. \quad (29)$$

The default local file is `<project-root>/char_decomposition_cache.db`. Python's `sqlite3` stores both full character decompositions and final singlet integers. The schema has `user_version=1`, independently of the MySQL theory database. There is no combined product-group decomposition table; identical simple factors reuse the same entries.

Table	Columns and primary key
<code>character_decompositions</code>	<code>algebra</code> TEXT, <code>dynkin_labels</code> TEXT, <code>adams_powers</code> TEXT, <code>adams_order</code> INTEGER, <code>terms_json</code> TEXT. Key: <code>(algebra, dynkin_labels, adams_powers)</code> .
<code>singlet_coefficients</code>	<code>algebra</code> TEXT, <code>product_key</code> TEXT, <code>coefficient</code> TEXT. Key: <code>(algebra, product_key)</code> .

All columns are non-null, and both tables use `WITHOUT ROWID`. Label and power tuples are compact JSON text; signed coefficients are decimal strings, avoiding SQLite's 64-bit integer limit [9]. Decomposition payloads are lists of `[labels, "coefficient"]` pairs. The full power tuple is required: total Adams order alone does not distinguish, for example, $R^{\otimes 2}$ from $\psi^2(R)$.

Symbolic singlet keys contain `["characters", canonical_product]`. Repeated base labels merge by adding exponents; factors sort in descending order. Already-decomposed inputs use a separate `"decompositions"` namespace. Zeros and negative coefficients are valid cache hits. Returned decompositions are copies, so caller mutation does not change a cached entry.

8.1 Lookup order and concurrency

A singlet lookup checks thread-local memory, then SQLite, before requesting any partial decomposition. Same-base-irrep partial products use persistent entries; mixed partial decompositions live only in the thread-local `partial_products` dictionary. Only the final complete-product scalar is persisted as a singlet.

Each thread owns a connection. PID checks avoid inherited connections; before starting a process pool the parent closes its connection. Workers calculate independent requests, and the parent batches completed writes by dependency level. Sequential batches flush after 64 entries and at completion. LiE work occurs outside write transactions. Failed calculations do not cache a guessed zero.

WAL allows concurrent readers and a writer, while writers remain serialized; it assumes a local filesystem [8]. The code uses a 30-second busy timeout and bounded retries. Python connection contexts manage transactions; this cache's context manager additionally closes the connection and clears thread-local entries [10].

Legacy JSON file discovery/import and `cache_path()` are removed. The new projection preserves existing SQLite keys and schema; no migration is needed. FORM still performs the fugacity expansion on every run.

9 Python functions and their mathematical roles

Function or class	Mathematical/computational role
<code>_parse_input</code>	Resolves the ordered factors in Eq. (1) and validated hypermultiplet data.
<code>_matter_character_multiplicities</code>	Constructs Eq. (10), omits trivial factor characters, and applies conjugation as in Eq. (11).
<code>_character_basis</code>	Assigns each pair (a, R) one FORM function C_p and records vector and matter character monomials.
<code>_build_form_program</code>	Implements Eqs. (7), (17), (14), and (20).
<code>run_form</code>	Shared helper in <code>common/form_utils.py</code> ; runs FORM in an isolated temporary directory and requires a successful, clean process result.
<code>_parse_form_output</code>	Converts each exact FORM monomial to a <code>FormTerm</code> ; rational coefficients become Python <code>Fraction</code> objects.
<code>get_decomposition,</code> <code>get_decompositions</code>	Retrieve or construct Eq. (21); batch missing prerequisites by tensor-factor count.
<code>get_singlet_multiplicities</code>	Look up symbolic-product singlets first; calculate missing integers with <code>_character_singlets</code> .
<code>singlet_multiplicities</code>	Cache singlets from already-decomposed inputs; misses call <code>_compute_singlet_multiplicities</code> .
<code>_split_character_product</code>	Greedily split individual Adams factors using Eq. (28c). Repeated factors may separate.
<code>_character_singlets</code>	Internal <code>plan()</code> identifies needed parts; <code>expand()</code> builds and memoizes mixed parts before final pairing.
<code>_dual_permutation,</code> <code>_singlet_pair</code>	Cache the label permutation in Eq. (28d) and evaluate Eq. (28b), including signed coefficients.
<code>_tensor_decompositions</code>	LiE decomposition of intermediate products; zero and scalar-trivial factors are handled directly.
<code>_compute_singlet_multiplicities</code>	For decomposed inputs, sort by support size, recursively build two parts and pair dual labels.
<code>_project_terms</code>	Multiplies factorwise singlet multiplicities and collects equal powers of t, y, u .
<code>_to_sage_polynomial</code>	Returns an exact flat Laurent polynomial in $\mathbb{Q}[t^{\pm 1}, y^{\pm 1}, u^{\pm 1}]$; performs no representation calculation.
<code>calculate_index</code>	Coordinates the complete public in-memory calculation.
<code>calculate_index_from_file</code>	Loads JSON and delegates to <code>calculate_index</code> .

10 JSON examples and command-line use

A simple-factor theory retains the existing form:

```
{
  "algebra": "A2",
  "hypermultiplets": [
    {"representation": "fundamental", "number": 6, "kind": "full"}
  ]
}
```

A product theory names every gauge factor. Missing entries in `representations` mean singlets:

```
{
  "gauge_groups": [
    {"id": "left", "algebra": "A1"},
    {"id": "right", "algebra": "A1"}
  ],
  "hypermultiplets": [{
    "representations": {
      "left": "fundamental", "right": "fundamental"
    },
    "number": 2, "kind": "full"
  }]
}
```

With the Sage environment active and `form` and `lie` on `PATH`:

```
python -m index.n2_theory_index theory.json --order 10
```

The default cache is `char_decomposition_cache.db` at the project root. Use `--cache-database /path/to/cache.db` for an explicit file, or `--cache-directory /path/to/directory` for the default filename inside that directory. These options are mutually exclusive; the corresponding Python arguments are `database_path=` and `cache_directory=`.

For example, an unlimited single-worker calculation through t^{18} is

```
result = calculate_index(data, 18, processes=1, timeout=None,
                        database_path="/tmp/fresh_index_cache.db")
```

Use a previously nonexistent database for a cold-cache timing. `timeout=None` is passed through to `FORM` and `LiE`; it removes their subprocess timeout.

11 Validation and representative performance

11.1 Earlier backend replacement checks (1 September snapshot)

The former sparse Sage weight-lattice implementation was retained long enough to serve as an independent reference before replacement. Exact comparisons gave:

Theory	Order	Check
$SU(2)$ with four full fundamentals	t^{18}	exact agreement
$SU(3)$ with six full fundamentals	t^{18}	exact agreement
$SU(4)$ with eight full fundamentals	t^{18}	exact agreement
$SU(2) \times SU(2)$ bifundamental theory	t^{10}	independent exact agreement
Decoupled product theories	t^6	factorization identity
$SU(2)^3$ trifundamental half hyper	t^6	factor-order invariance

For conformal $SU(3)$ SQCD, the first nonzero terms are

$$\begin{aligned} \mathcal{I} = 1 + t^4 \left(\frac{36}{u^2} + u^4 \right) - t^5 u^2 (y + y^{-1}) \\ + t^6 \left(\frac{40}{u^3} - 36 + u^6 \right) + O(t^7). \end{aligned} \quad (30)$$

Representative product-group timings through t^{10} were:

Theory	Cold cache	Warm cache
$A_1 \times A_1$ bifundamental	1.030 s	0.042 s
$A_1 \times G_2$ mixed matter	0.274 s	0.127 s

Timings are machine-dependent. They illustrate that the persistent decomposition cache removes repeated LiE work but not FORM’s per-run series expansion. The complete project test run after that earlier replacement passed 55 tests; two live MySQL integration tests were skipped because no test database was requested.

11.2 SQLite and sparse-projection checks (9 September)

The temporary projection candidate matched all seven comparative index cases and 319 singlet queries against the previous SQLite implementation (a9b7009). Of these, 202 also matched independent $SU(2)/SU(3)$ weight calculations. All 216 dual-label comparisons across 18 Cartan types matched Sage. All 4,996 old SQLite singlet entries tested were reused without recalculation. These checks preceded promotion to the implementation now committed as 0a402ec.

The full post-projection suite ran 170 tests in 11.461 s: 168 passed and two live MySQL tests were skipped. Seven new regressions cover signed complex/pseudoreal pairing, grouped Adams splitting, mixed products, independent weights, parallel generation, trivial factors and zeros. These are local validation results; they do not establish correctness at all ranks and cutoffs.

11.3 Measured speedups and unlimited A32 profile

Cold calculation	Previous (s)	Revised (s)
A_8, t^{18} , singlet projection	4.599	0.849
A_{16}, t^{12} , singlet projection	1.473	0.207
A_{32}, t^8 , singlet projection	0.469	0.117
A_8, t^{18} , complete index	6.213	2.443

These are three-trial medians with fresh cache files and one worker, for SQCD with $2(r+1)$ full fundamental hypers. Projection excludes the shared FORM expansion. Warm A8 complete-index time remained about 1.70 s. Four-worker comparisons also matched, though process startup can slow small cases.

On 10 September, $A_{32} = SU(33)$ with 66 full fundamental hypers completed through t^{18} with no timeout and one worker:

Stage	Cold (s)	Warm (s)
Input/validation/algebra initialization	4.483	0.009
FORM expansion	1.357	1.406
FORM output parsing	0.212	0.239
Singlet projection, including cache work	88.229	0.127
Complete calculation	94.293	1.786

One LiE call decomposing $\psi^3(\text{adj})^2$ took 82.699 s, or 87.7% of the cold total. Its inputs each have 10 irrep terms and its output has 91. All other LiE calls together took 5.066 s. The warm run made no LiE calls. Both index results contain 117 nonzero monomials and match exactly. This is one wall-clock profile, not a median; the warm run shares Sage initialization but uses a new cache client.

Evidence and reproduction commands are in `output/benchmarks/singlet_projection_benchmark.md` and `output/benchmarks/a32_unlimited/report.md`. The unlimited run shows that the remaining cost is an intermediate full decomposition, not the final sparse pairing.

12 Interpretation and limitations

- The result is a signed protected index, not a complete operator spectrum.
- Flavor fugacities are unrefined. Matter multiplicities are integer coefficients.
- The code supports semisimple products of the listed finite simple algebras, but not $U(1)$ gauge factors.
- The Lie algebra does not determine the global form of the gauge group, discrete theta data, or the spectrum of line operators.
- Input validation checks representation syntax and physical admissibility of a half hyper. The index routine does not require the beta functions or global gauge anomalies to vanish; those are separate checker outputs.
- High orders still produce many FORM monomials and expensive intermediate LiE products. The cache reduces repeated work, and sparse pairing avoids the complete product's decomposition; neither removes all combinatorial growth or guarantees an optimal intermediate split.

References

- [1] A. Gadde, E. Pomoni, and L. Rastelli, “The Veneziano Limit of $\mathcal{N} = 2$ Superconformal QCD: Towards the String Dual of $\mathcal{N} = 2$ $SU(N_c)$ SYM with $N_f = 2N_c$,” JHEP 03 (2010) 032, [arXiv:0912.4918](#).
- [2] S. Benvenuti, B. Feng, A. Hanany, and Y.-H. He, “Counting BPS Operators in Gauge Theories: Quivers, Syzygies and Plethystics,” JHEP 11 (2007) 050, [arXiv:hep-th/0608050](#).
- [3] M. A. A. van Leeuwen, A. M. Cohen, and B. Lisser, *LiE, A Package for Lie Group Computations*, manual, 1992. In particular, Secs. 3.4 and 4.5 describe Adams operations and tensor-product decomposition polynomials. [CWI manual record](#).
- [4] J. Kuipers, J. A. M. Vermaseren, and collaborators, *FORM 5.0.0 Reference Manual*, 2026.
- [5] J. Kuipers, T. Ueda, J. A. M. Vermaseren, and J. Vollinga, “FORM version 4.0,” Comput. Phys. Commun. 184 (2013) 1453–1467, [arXiv:1203.6543](#).
- [6] P. Etingof, *Lie Groups and Lie Algebras*, MIT 18.755 lecture notes (2024), Proposition 32.1 and Corollary 35.9. [Official MIT notes](#).
- [7] The Sage Developers, *Weyl character rings*, `dual()` and `inner_product()`. [Official Sage documentation](#).
- [8] SQLite, *Write-Ahead Logging*, §2.2. [Official WAL documentation](#).
- [9] SQLite, *Datatypes In SQLite*, §2. [Official datatype documentation](#).
- [10] Python Software Foundation, *sqlite3: DB-API 2.0 interface for SQLite databases*. [Official Python documentation](#).

New references were checked on 10 September 2026. The splitting heuristic, function map and measurements are derived from the project source and local validation, rather than attributed to the representation-theory literature.